

ALGORITHM RESEARCH & PRESENTATION PROJECT

Group Assignment — Algorithm Theory, Flowcharts & Pseudocode (No Coding Required)

So far, you've learned how to break down algorithms, sketch them as flowcharts, and write them out in pseudocode. Now, it's your turn to pick a well-known algorithm and show what you've learned. Don't stress about coding Python and data types which we haven't learned yet. Just focus on explaining the logic clearly, using words, diagrams, and some simple numbers if you need them.

There are 27 students in our class, split into 6 groups of 3 or 4 or 5 people each. If somebody doesn't prefer group work it's okay you can work alone, but you still have to pick one of the given topics and come up with your own opening question for your presentation. These sections we are not learning about code for now you are focused on understanding and explaining the logic.

What Every Group Must Produce

Regardless of your topic, every group's research and presentation must include the following five parts:

1. A short real-world explanation of the problem the algorithm solves (**use the guiding hook question**).
2. A hand-drawn or digital flowchart showing the full logic, using standard flowchart symbols (start/end, process, decision, input/output etc which we have learned).
3. Pseudocode written in plain structured English/steps general logic only, no Python or any programming language syntax.

Group 1: Linear Search Algorithm

Hook: "How does a phone find a contact by scrolling through a list one by one?"

Research questions to answer:

- When do you use linear search in the real world?
- How does it check items until it finds a match (or not)?
- What's the best, worst, and usual case?
- Why is it simple but slow on big lists?

Group 2: Binary Search Algorithm

Hook: "How does a dictionary let you find a word without reading every page?"

Research questions to answer:

- What's required before you run binary search?
- How does it cut the search in half again and again?
- How many comparisons does it use, compared to linear search?
- Share a non-computer example that works like binary search.

Group 3: Bubble Sort Algorithm

Hook: "How would you sort a row of students by height by only swapping two neighbors?"

Research questions to answer:

- How does it compare and swap neighbors to sort things out?
- Why do you need to repeat passes?

- “Largest value bubbles up” — what does that really mean?
- Why is it easy to learn but bad for big lists?

Group 4: Selection Sort Algorithm Topic for Assignment

Hook: “If you had to sort cards by always picking the smallest one left, how would you do it?”

Research questions to answer:



- How does it find the smallest value each round?
- How is it different from bubble sort, since both do rounds?
- Why does selection sort make fewer swaps?
- Give a real example where fewer swaps matter.
- Team Member: Rothana, Dara, Deth, lyheng, Samnang, Deadline next week

Group 5: Recursion Algorithm (Factorial & Fibonacci)

Hook: “Why can a function that calls itself solve the problem by handling smaller versions?”

Research questions to answer:

- What’s a base case? Why does every recursive function need it?
- How does factorial(n) break down to factorial(n-1)?
- How does Fibonacci use two earlier results to get the next?
- What happens if you never reach the base case?

Notes: No Python code is needed or related for this project just to focus entirely on the logic. You will learn to turn this same logic into real Python code